

Corrigé FramPict

A - Réutilisation (5 pts)

1.a) Listing 3

Polymorphisme : **Forme** abstraite référence sous-classes. Extension sans modifier code client → principe Ouvert/Fermé.

1.b) Fonction d'ordre supérieur

drawDescription() délègue à **getDescription()** abstraite (fournie par sous-classes). Pattern Template Method.

1.c) Paramétrage

- **Type** : type statique **Forme**, type dynamique **Cercle/Ligne** (polymorphisme)
- **Paramètre** : **getDescription()** (point d'extension)
- **Argument** : redéfinition concrète dans **Cercle/Ligne**

2. Classe Ligne

```
public class Ligne extends Forme {
    private Point debut, fin;
    public Ligne(Point debut, Point fin) {
        this.debut = debut; this.fin = fin;
    }
    @Override
    public String getDescription() {
        return "- une ligne de " + debut + " à " + fin;
    }
}
```

3. Tests

- Tests unitaires (JUnit) sur chaque sous-classe **Forme**
- Tests d'intégration sur **Dessin** (ajout, ordre, unicité via **Set**)
- Tests de non-régression lors extensions
- Séparer tests framework/base et extensions

B - Typage statique (6 pts)

1. equals dans Ligne

Dans Forme :

```

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null) return false;
    return obj.equalsTo(this);
}

public abstract boolean equalsTo(Forme other);
public abstract int getId();

```

Dans Ligne :

```

private static final int TYPE_ID = 1;

@Override
public int getId() { return TYPE_ID; }

@Override
public boolean equalsTo(Forme other) {
    if (other.getId() != TYPE_ID) return false;
    Ligne l = (Ligne) other;
    return (debut.equals(l.debut) && fin.equals(l.fin));
}

```

2. Liskov

Précondition non renforcée (accepte `Object`), postcondition non affaiblie (équivalence), comparaison valeurs.

3. f1.drawDescription()

1. `drawDescription()` dans `Forme`
2. `getDescription()` dans `Ligne` (polymorphisme)

4. Vector v = new Vector();

Non accepté : génériques invariants. Violerait sécurité type.

5. Factoriser equals dans Forme

Solution : `equals` factorisé dans `Forme` (appelle `equalsTo` via double dispatch). Chaque sous-classe redéfinit `equalsTo(Forme)` et `getId()`. Évite `instanceof`/`getClass()`.

C - Dessins arborescents (4 pts)

1. Pattern Composite

- `Dessin` extends `Forme`
- `contenu`: `List<Forme>`

Code:

```
public class Dessin extends Forme {
    private List<Forme> contenu = new ArrayList<>();
    private String title;

    public void add(Forme f) { contenu.add(f); }

    @Override
    public String getDescription() {
        return "Un dessin de " + title + " obtenu par :";
    }

    @Override
    public void drawDescription() {
        System.out.println(getDescription());
        for (Forme f : contenu) {
            System.out.print("-");
            f.drawDescription();
        }
    }
}
```

2. dessinsInclus()

Référence inverse : `Forme` garde `List<Dessin> parents`. Dans `Dessin.add(Forme f)`, ajouter `this` aux parents de `f`.

```
public Collection<Dessin> dessinsInclus() {
    return Collections.unmodifiableList(parents);
}
```